Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información
y Ciencias de la Computación

ER-AFD / DOCUMENTO 02 DE 03

Casos de prueba

Convertidor de expresiones regulares a AFD

Construcción explícita ER → AFND → AFD

PRESENTADO POR

SELVIN KLEBER GIRÓN ALVAREZ	**Carné: 3190-15-15973**
JACKELIN PAULINA AVILA CIFUENTES	**Carné: 6590-24-772**
JOSE JULIO JIMENEZ MATUL	**Carné: 6590-24-13855**
MARCOS RODOLFO ARANA APOLINAR	**Carné: 1490-22-16405**

Guatemala, 20 de octubre de 2026

Proyecto del curso · Implementación en Python

1. Método y entorno de prueba

El objetivo fue comprobar aceptación, rechazo, precedencia de operadores, clases, escapes y tratamiento de entradas inválidas. Se compararon los resultados con valores esperados y se verificaron los códigos reales devueltos por los procesos.

Dato	Valor registrado
Fecha	20 de octubre de 2026
Sistema	Windows 11, compilación 26300
Python	3.12.10
Archivo de pruebas	pruebas.py; biblioteca estándar unittest
Comando automatizado	python -X utf8 pruebas.py

Cobertura registrada

Grupo	Cantidad	Resultado
Métodos automatizados	10	10 correctos; suite OK.
ER de casos funcionales	42	152 evaluaciones dirigidas coincidentes.
ER inválidas	24	Rechazadas con ValueError y posición.
Combinaciones de cadenas	1,023	Coinciden con las condiciones de referencia.
Complementos ASCII	256	Coinciden con el complemento esperado.

Las categorías son verificaciones diferentes y no deben sumarse como si fueran expresiones distintas. Los porcentajes de coincidencia se limitan a estos casos.

Cómo se obtuvieron las capturas

Se ejecutaron procesos reales de Python y se guardaron stdout, stderr y el código de salida. Las imágenes son capturas de una vista de lectura en navegador de esos registros; no son capturas de una terminal Linux. Algunas vistas separan AFND y AFD para mejorar la lectura. Los registros completos se conservan en doc/evidencias.

2. Aceptación: construcción del AFND

ER	Cadena	Esperado	Obtenido
(a|b)*abb	aabb	Aceptada, código 1	Aceptada, código 1

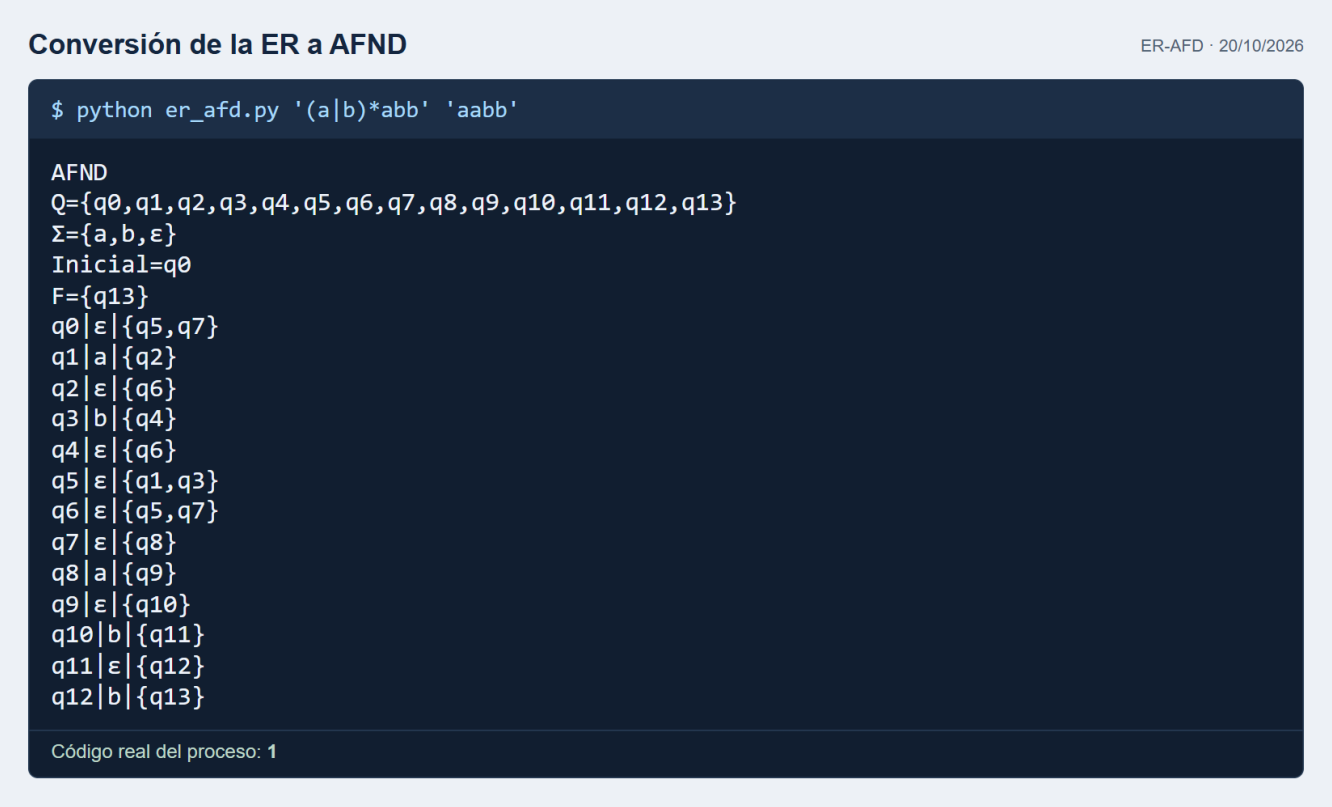


Figura 1. Primera parte del registro de aceptación: estados, alfabeto, final y transiciones del AFND.

El AFND tiene 14 estados y conserva las transiciones ε introducidas por la unión, la cerradura y las concatenaciones. q0 es inicial y q13 es final.

El archivo aceptacion.txt contiene la salida completa de esta misma ejecución, incluida la conversión a AFD y la evaluación que continúa en la siguiente página.

3. Aceptación: AFD y recorrido

La ER y la cadena son las mismas del caso anterior. El AFD tiene cinco estados alcanzables y el único estado final es Q4.

Conversión a AFD y cadena aceptada ER-AFD · 20/10/2026

```
$ python er_afd.py '(a|b)*abb' 'aabb'

AFD
Q={Q0,Q1,Q2,Q3,Q4}
Σ={a,b}
Inicial=Q0
F={Q4}
Q0|a|Q1
Q0|b|Q2
Q1|a|Q1
Q1|b|Q3
Q2|a|Q1
Q2|b|Q2
Q3|a|Q1
Q3|b|Q4
Q4|a|Q1
Q4|b|Q2

Recorrido
Q0|a|Q1
Q1|a|Q1
Q1|b|Q3
Q3|b|Q4
Cadena aceptada
```

Código real del proceso: **1**

Figura 2. Segunda parte del mismo registro: matriz del AFD y recorrido de aabb.

El recorrido observado es Q0 → Q1 → Q1 → Q3 → Q4. La cadena se consume completamente y termina en Q4; por eso se acepta y el proceso devuelve 1.

4. Rechazo de una cadena

ER	Cadena	Esperado	Obtenido
(a|b)*abb	abba	Rechazada, código 0	Rechazada, código 0

Cadena rechazada por el AFD

ER-AFD · 20/10/2026

```
$ python er_afd.py '(a|b)*abb' 'abba'

AFD
Q={Q0,Q1,Q2,Q3,Q4}
Σ={a,b}
Inicial=Q0
F={Q4}
Q0|a|Q1
Q0|b|Q2
Q1|a|Q1
Q1|b|Q3
Q2|a|Q1
Q2|b|Q2
Q3|a|Q1
Q3|b|Q4
Q4|a|Q1
Q4|b|Q2

Recorrido
Q0|a|Q1
Q1|b|Q3
Q3|b|Q4
Q4|a|Q1
Cadena rechazada
```

Código real del proceso: **0**

Figura 3. La cadena abba termina en Q1, que no es final.

El autómata pasa por Q4 después de leer abb, pero todavía falta la última a. Al consumirla avanza a Q1. El estado final se comprueba al terminar toda la entrada, por lo que el resultado correcto es rechazo.

Registro completo: doc/evidencias/rechazo.txt.

5. Cadena vacía y épsilon

ER	Cadena	Esperado	Obtenido
ε	Vacía: ""	Aceptada, código 1	Aceptada, código 1

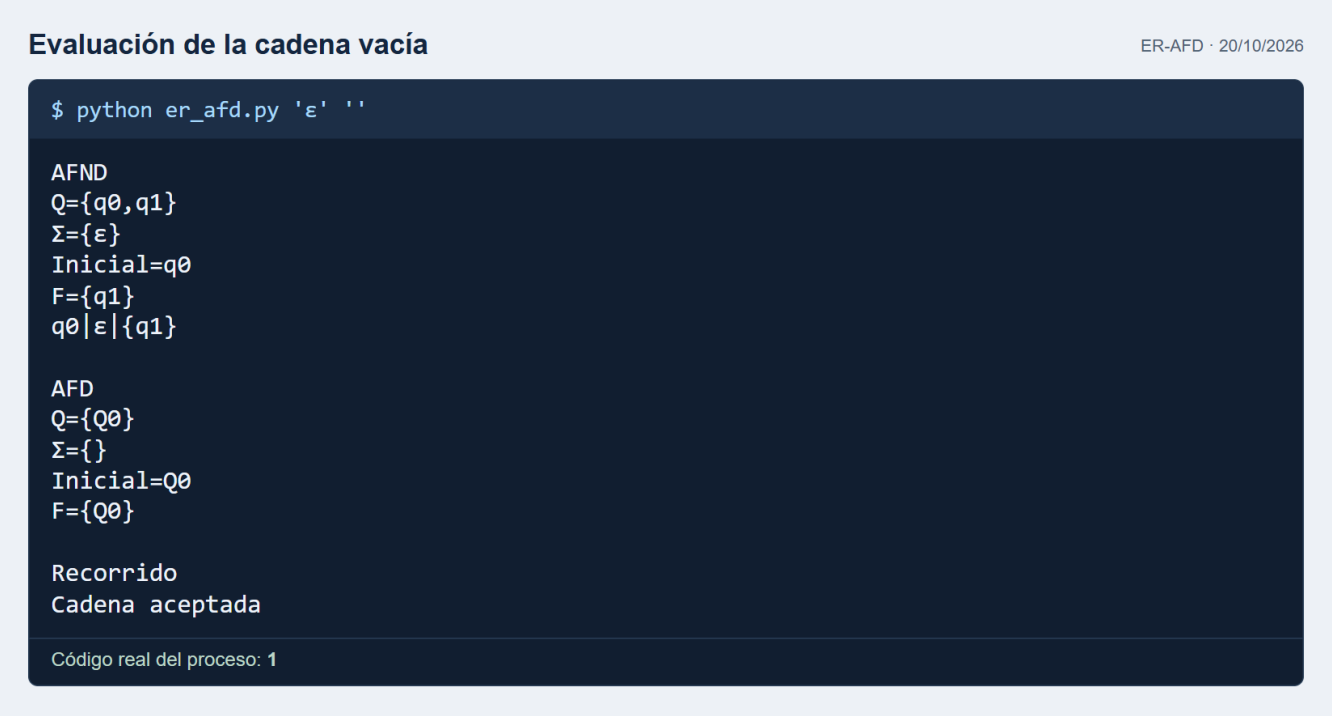


Figura 4. El cierre ε de q0 alcanza el final, por lo que Q0 ya es un estado de aceptación.

No se consume ningún carácter y no hay filas de recorrido. La presencia de ε en la ER indica una transición sin consumo; no significa que deba escribirse la letra e en la cadena.

Otros casos vacíos comprobados

Expresión	Cadena	Resultado esperado y obtenido
a*	""	Aceptada.
ab	""	Rechazada.
(ε|a)b	b	Aceptada.

6. Punto y literal coincidentes

Caso: ER **.b|ac**, cadena **ac**. Se esperaba aceptación y se obtuvo aceptación con código 1. La transición inicial debe considerar tanto el punto como la letra a.

Punto y literales: clases sin solapamientos

ER-AFD · 20/10/2026

```
$ python er_afd.py '.b|ac' 'ac'

AFD
Q={Q0,Q1,Q2,Q3,Q4,Q5}
Σ={.,b,a,c}
Inicial=Q0
F={Q3,Q5}
Clases de entrada={b,a,c,[^a-c]}
La tabla usa clases disjuntas para evitar coincidencias entre símbolos.
Q0|b|Q1
Q0|a|Q2
Q0|c|Q1
Q0|[^a-c]|Q1
Q1|b|Q3
Q1|a|Q4
Q1|c|Q4
Q1|[^a-c]|Q4
Q2|b|Q3
Q2|a|Q4
Q2|c|Q5
Q2|[^a-c]|Q4
Q3|b|Q4
Q3|a|Q4
Q3|c|Q4
Q3|[^a-c]|Q4
Q4|b|Q4
Q4|a|Q4
Q4|c|Q4
Q4|[^a-c]|Q4
Q5|b|Q4
Q5|a|Q4
Q5|c|Q4
Q5|[^a-c]|Q4

Recorrido
Q0|a|Q2
Q2|c|Q5
Cadena aceptada
```

Código real del proceso: **1**

Figura 5. La tabla muestra clases disjuntas y acepta ac mediante Q0 → Q2 → Q5.

Registro completo: doc/evidencias/solapamiento.txt. También se verificó que aa y bc sean rechazadas para esta misma ER.

7. Entradas inválidas y respuesta

Estos casos deben detenerse con un mensaje entendible y código 2. No deben confundirse con una cadena válida que simplemente no pertenece al lenguaje.

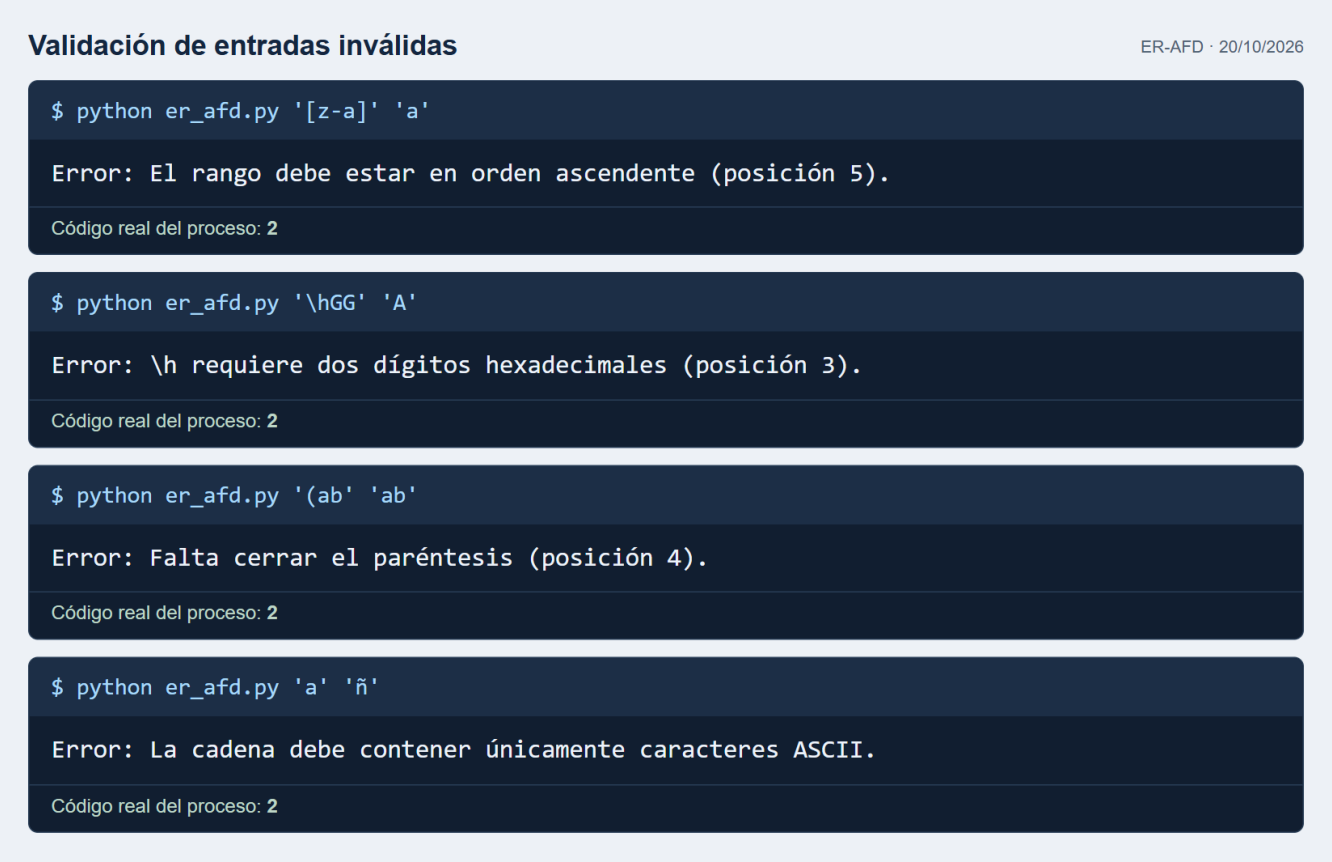


Figura 6. Errores reales de rango, hexadecimal, paréntesis y entrada no ASCII.

Entrada	Motivo	Resultado
[z-a]	Rango descendente.	Error controlado; código 2.
\hGG	Dos caracteres que no son dígitos hexadecimales.	Error controlado; código 2.
(ab	Paréntesis sin cerrar.	Error controlado; código 2.
ER a; cadena ñ	La cadena no pertenece a ASCII.	Error controlado; código 2.

Además, la suite verificó argumentos faltantes, un grupo de 300 paréntesis anidados, escapes incompletos, conjuntos vacíos y extremos de rango inválidos. Los escenarios de consola comprobados no mostraron un traceback al usuario.

8. Casos por operador y escape

En cada fila se comprobó que la primera cadena se acepte y la segunda se rechace. Todos los resultados de esta selección coincidieron con lo esperado.

Expresión regular	Cadena aceptada	Cadena rechazada
ab	ab	abc
a|bc	bc	ac
(a|b)c	ac	abc
a*	"" (vacía)	ab
[abc]	b	d
[^abc]	d	a
[a-z][0-9]	a0	A0
[a-zA-Z]*	Hola	hola1
*	*	a
\p	p	q
\\	\	/
\.	.	a
\N\N*	12345	1a
\h20	" " (espacio)	20
\h41	A	a
\A\A*	HOLA	Hola
\a\a*	hola	Hola
\z\z*	Hola	Hola1
hola\ mundo	hola mundo	holamundo

La selección se complementa con las pruebas de punto, épsilon, combinaciones y entradas inválidas presentadas en las demás páginas. Los 152 resultados dirigidos se conservan en ejecuciones.json.

9. Batería automatizada y cierre

Resultado de la batería automatizada ER-AFD · 20/10/2026

```
$ python -X utf8 pruebas.py

test_cadenas (__main__.PruebasAutomatas.test_cadenas) ... ok
test_clases_sin_solapamientos (__main__.PruebasAutomatas.test_clases_sin_solapamientos)
... ok
test_combinaciones (__main__.PruebasAutomatas.test_combinaciones) ... ok
test_complementos_ascii (__main__.PruebasAutomatas.test_complementos_ascii) ... ok
test_consola (__main__.PruebasAutomatas.test_consola) ... ok
test_consola_cadena_vacia_y_guion
(__main__.PruebasAutomatas.test_consola_cadena_vacia_y_guion) ... ok
test_consola_codificacion_windows
(__main__.PruebasAutomatas.test_consola_codificacion_windows) ... ok
test_consola_errores (__main__.PruebasAutomatas.test_consola_errores) ... ok
test_expresiones_invalidas (__main__.PruebasAutomatas.test_expresiones_invalidas) ...
ok
test_orden_alfabeto (__main__.PruebasAutomatas.test_orden_alfabeto) ... ok

----------------------------------------------------------------------
Ran 10 tests in 1.365s

OK

Código real del proceso: 0
```

Figura 7. Ejecución real de pruebas.py: diez métodos correctos y código de salida 0.

La suite comprueba casos dirigidos, errores de sintaxis, orden del alfabeto, clases disjuntas, complementos ASCII, combinaciones, lanzadores, códigos de salida y codificación de consola. Para repetirla desde la raíz del proyecto:

```
python pruebas.py
```

Se corrigieron dos fallos detectados durante las verificaciones previas: la salida de símbolos griegos con una codificación de Windows incompatible y el traceback por exceso de anidamiento. La suite incluye casos que vuelven a comprobar ambos comportamientos.

La verificación corresponde a Windows y Python 3.12.10. No se presenta como prueba de rendimiento ni como validación del Makefile en Linux. Las imágenes y registros se entregan en capturas/ y evidencias/ para permitir su revisión.